

ESP32 Accident Detection & Alert System — Source Code

GitHub documentation copy. Credentials are intentionally kept as placeholders. Do not replace them with real secrets before uploading.

```
/*
  ESP32 Accident Detection & Alert System
  -----
  Components:
  - ESP32
  - MPU6050 Accelerometer/Gyroscope
  - NEO-6M GPS
  - Telegram Bot
  - Buzzer (optional)

  Function:
  1. ESP32 connects to Wi-Fi.
  2. MPU6050 continuously measures acceleration.
  3. If acceleration exceeds the configured threshold,
     a possible accident is detected.
  4. GPS location is obtained.
  5. Telegram Bot sends an emergency notification
     containing the location.

  IMPORTANT:
  - Replace Wi-Fi credentials and Telegram credentials.
  - Never upload real credentials to GitHub.
  - Calibrate the accident threshold using your actual hardware.
*/

#include <WiFi.h>
#include <WiFiClientSecure.h>
#include <UniversalTelegramBot.h>
#include <Wire.h>
#include <MPU6050.h>
#include <TinyGPSPlus.h>

// =====
// Wi-Fi Configuration
// =====

const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

// =====
// Telegram Configuration
// =====

#define BOT_TOKEN "YOUR_TELEGRAM_BOT_TOKEN"
#define CHAT_ID "YOUR_TELEGRAM_CHAT_ID"

WiFiClientSecure client;
UniversalTelegramBot bot(BOT_TOKEN, client);

// =====
// MPU6050
// =====

MPU6050 mpu;

// =====
// GPS
// =====

TinyGPSPlus gps;

// GPS pins
// Change these according to your actual wiring.
#define GPS_RX_PIN 16
#define GPS_TX_PIN 17

HardwareSerial GPS_Serial(2);

// =====
// Buzzer
// =====

#define BUZZER_PIN 25

// =====
// Accident Detection
// =====

// Approximate threshold.
// MUST be calibrated according to your project.
const float ACCIDENT_THRESHOLD_G = 2.5;

// Prevent repeated alerts
bool accidentDetected = false;

// =====
// Functions
// =====

void connectWiFi()
```

```

{
    Serial.print("Connecting to Wi-Fi");

    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

    while (WiFi.status() != WL_CONNECTED)
    {
        delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("Wi-Fi connected!");
    Serial.print("ESP32 IP Address: ");
    Serial.println(WiFi.localIP());
}

// -----

void readGPS()
{
    while (GPS_Serial.available() > 0)
    {
        gps.encode(GPS_Serial.read());
    }
}

// -----

String getGPSLocation()
{
    if (gps.location.isValid())
    {
        String latitude = String(gps.location.lat(), 6);
        String longitude = String(gps.location.lng(), 6);

        String location =
            "Latitude: " + latitude +
            "\nLongitude: " + longitude;

        return location;
    }

    return "GPS location not available.";
}

// -----

void sendTelegramAlert()
{
    String message;

    message += "ACCIDENT ALERT\n\n";
    message += "Possible accident detected by ESP32.\n\n";

    message += "Location:\n";
    message += getGPSLocation();

    if (gps.location.isValid())
    {
        message += "\n\nGoogle Maps:\n";
        message += "https://maps.google.com/?q=";
        message += String(gps.location.lat(), 6);
        message += ",";
        message += String(gps.location.lng(), 6);
    }

    message += "\n\nPlease check immediately.";

    bool sent = bot.sendMessage(CHAT_ID, message, "");

    if (sent)
    {
        Serial.println("Telegram alert sent successfully.");
    }
    else
    {
        Serial.println("Failed to send Telegram alert.");
    }
}

// -----

float getAccelerationMagnitude()
{
    int16_t ax;
    int16_t ay;
    int16_t az;

    mpu.getAcceleration(&ax, &ay, &az);

    // MPU6050 default sensitivity:
    // ±2g = approximately 16384 LSB/g

    float ax_g = ax / 16384.0;

```

```

float ay_g = ay / 16384.0;
float az_g = az / 16384.0;

float magnitude =
    sqrt(
        (ax_g * ax_g) +
        (ay_g * ay_g) +
        (az_g * az_g)
    );

return magnitude;
}

// -----

void activateBuzzer()
{
    digitalWrite(BUZZER_PIN, HIGH);

    delay(2000);

    digitalWrite(BUZZER_PIN, LOW);
}

// =====
// SETUP
// =====

void setup()
{
    Serial.begin(115200);

    // Buzzer
    pinMode(BUZZER_PIN, OUTPUT);
    digitalWrite(BUZZER_PIN, LOW);

    // I2C
    Wire.begin();

    // MPU6050 initialization
    Serial.println("Initializing MPU6050...");

    mpu.initialize();

    if (mpu.testConnection())
    {
        Serial.println("MPU6050 connected successfully.");
    }
    else
    {
        Serial.println("MPU6050 connection failed!");
    }

    // GPS initialization
    GPS_Serial.begin(
        9600,
        SERIAL_8N1,
        GPS_RX_PIN,
        GPS_TX_PIN
    );

    // Wi-Fi
    connectWiFi();

    // Telegram HTTPS
    client.setInsecure();

    Serial.println();
    Serial.println("-----");
    Serial.println("Accident Detection System Ready");
    Serial.println("-----");
}

// =====
// LOOP
// =====

void loop()
{
    // Continuously read GPS
    readGPS();

    // Read acceleration
    float acceleration = getAccelerationMagnitude();

    Serial.print("Acceleration: ");
    Serial.print(acceleration);
    Serial.println(" g");

    // Accident detection
    if (acceleration >= ACCIDENT_THRESHOLD_G)
    {
        Serial.println("Possible accident detected!");

        activateBuzzer();
    }
}

```

```

// Send only one alert for this event
if (!accidentDetected)
{
    accidentDetected = true;

    // Give GPS a short opportunity to update
    unsigned long startTime = millis();

    while (millis() - startTime < 3000)
    {
        readGPS();
        delay(10);
    }

    sendTelegramAlert();
}

// Reset detection when acceleration returns to normal
if (acceleration < 1.5)
{
    accidentDetected = false;
}

delay(200);
}

```